

ARQUITECTURA DEL SISTEMA

SIGIOP-MOP

Sistema de Gestión de Inspecciones de Obras Públicas

Campo	Información
Organización	Duoc UC · Escuela de Informática y Telecomunicaciones
Proyecto	SIGIOP-MOP
Alcance inicial	SGII-DV · Dirección de Vialidad
Metodología de gestión	Kanban
Fecha	24/08/2026
Versión	Versión inicial
Estado	Versión inicial preparada para revisión formal

1. Datos del documento

1.1 Histórico de revisiones

Versión	Fecha	Descripción / cambio	Autor
1.0	24/08/2026	Versión inicial de Arquitectura del Sistema preparada para revisión formal y alineada con la línea base D01-D08.	Equipo SIGIOP-MOP

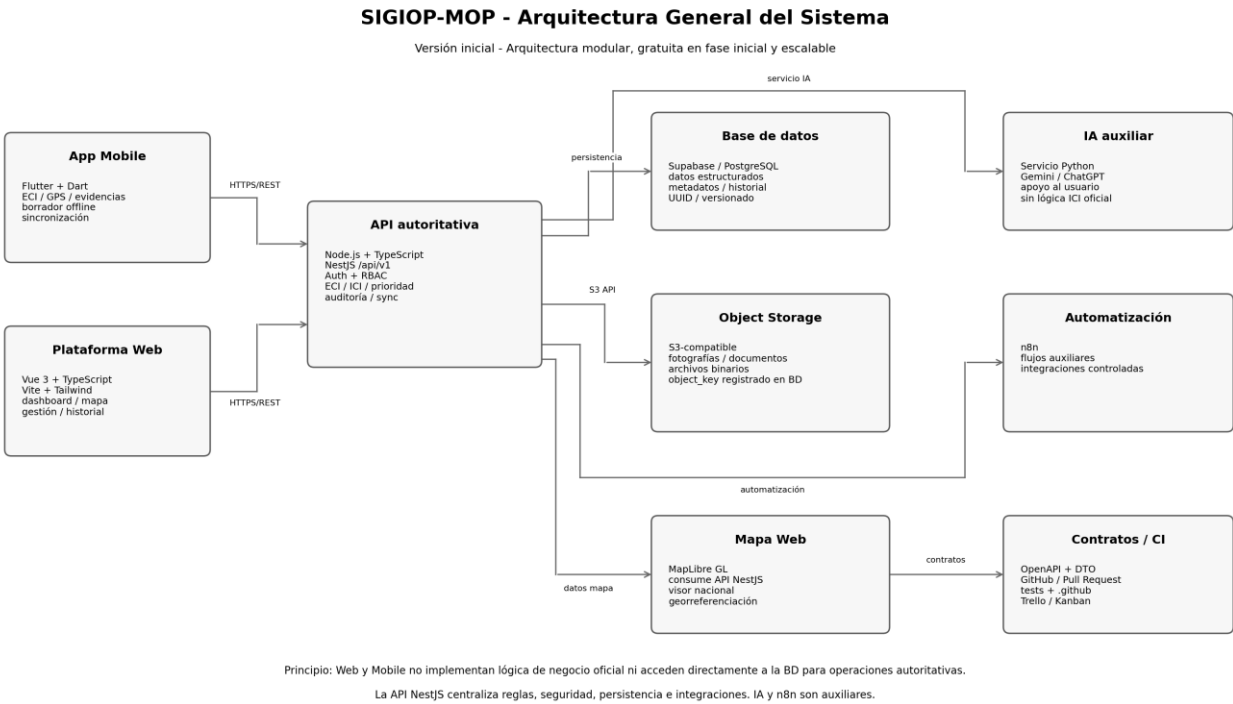
1.2 Propósito

Definir la arquitectura técnica propuesta para el MVP de SIGIOP-MOP, identificando clientes, backend, persistencia, almacenamiento de evidencias, componentes cartográficos, servicios auxiliares, seguridad, sincronización e integración. El documento toma como referencia el tipo de representación visual mostrado en los ejemplos de arquitectura entregados para Capstone, adaptándolo a la arquitectura real definida para SIGIOP-MOP.

2. Principios y restricciones arquitectónicas

- Fase inicial sin costo de licenciamiento o infraestructura para el proyecto académico, privilegiando planes gratuitos y componentes open source.
- Arquitectura modular y escalable, preparada para ampliar el alcance desde SGII-DV hacia otras unidades del MOP sin duplicar el núcleo transversal.
- Web y Mobile son clientes independientes de una API REST autoritativa.
- La lógica institucional, RBAC, cálculo ICI, prioridad, auditoría y sincronización se centralizan en el backend.
- Los datos estructurados y metadatos se separan de fotografías y otros archivos binarios.
- IA y n8n son componentes auxiliares y no sustituyen reglas institucionales ni decisiones profesionales.
- Las decisiones institucionales aún abiertas no se codifican por inferencia.

3. Arquitectura general



La arquitectura adopta un enfoque cliente-servidor modular. La aplicación móvil y la plataforma web consumen la misma API NestJS mediante HTTPS/REST. El backend actúa como Entry Point único para las operaciones autoritativas y coordina persistencia, almacenamiento de archivos, IA, automatización y servicios de consulta.

4. Componentes de la solución

Componente	Tecnología / herramienta	Responsabilidad
Aplicación móvil	Flutter + Dart	Trabajo en terreno: formulario ECI, respuestas, GPS, fotografías/evidencias, borradores y operación offline.
Plataforma web	Vue 3 + TypeScript + Vite + Tailwind CSS	Gestión: dashboard, mapa nacional, historial, reportes, seguimiento y administración.
API Backend	Node.js + TypeScript + NestJS	Entry Point REST /api/v1; autenticación, RBAC, reglas ECI/ICI, prioridad, auditoría, historial, sync e integraciones.
Base de datos	Supabase / PostgreSQL	Usuarios, roles, recintos, formularios, inspecciones, respuestas, ICI, historial, metadatos y trazabilidad.
Object Storage	S3-compatible	Fotografías, documentos y binarios. PostgreSQL conserva object_key y metadatos.
Mapa	MapLibre GL	Renderizado cartográfico en Web; consume datos georreferenciados desde la API NestJS.
IA auxiliar	Python + Gemini/ChatGPT	Apoyo a usuarios autorizados; desacoplada del cálculo ICI y de la modificación autónoma de datos oficiales.
Automatización	n8n	Flujos auxiliares e integraciones controladas; no contiene lógica institucional oculta.
Contratos	OpenAPI + DTO/schemas	Contrato versionado entre clientes y backend; modelos de request/response y errores.
Gestión técnica	GitHub + Trello	GitHub para repositorio, ramas, commits y Pull Requests; Trello para Kanban, prioridad, estado y tarjetas.

5. Flujo arquitectónico principal

Paso	Origen	Destino	Descripción
1	Usuario de terreno	App Mobile	Captura ECI, evidencia y GPS; almacena temporalmente cuando no existe conexión.
2	App Mobile	API NestJS	Envía operaciones mediante REST; sincronización batch con UUID e

			idempotency_key.
3	Usuario de gestión	Web	Consulta dashboard, mapa, historial, reportes y administración.
4	Web	API NestJS	Solicita datos y operaciones autorizadas; no implementa reglas oficiales en frontend.
5	API NestJS	Supabase/PostgreSQL	Lee/escrbe datos estructurados, estados, resultados y trazabilidad.
6	API NestJS	Object Storage	Gestiona referencias a fotografías/documentos mediante object_key.
7	API NestJS	IA / n8n	Invoca servicios auxiliares cuando corresponda.
8	API NestJS	Web / Mobile	Retorna respuesta consistente según contrato OpenAPI y permisos.

6. Arquitectura de datos y almacenamiento

Supabase/PostgreSQL mantiene la información estructurada del dominio. El almacenamiento de fotografías y documentos se desacopla mediante Object Storage S3-compatible para evitar que el crecimiento de evidencia binaria degrade o agote innecesariamente la capacidad de la base relacional.

Tipo de información	Ubicación	Ejemplos
Datos estructurados	PostgreSQL	usuarios, roles, recintos, inspecciones, respuestas, ICI, prioridades, estados, auditoría
Metadatos de evidencia	PostgreSQL	tipo, fecha, usuario, GPS, hash, object_key, relación con respuesta/criterio
Binarios	Object Storage	fotografías, PDF, documentos del recinto
Borradores offline	Dispositivo móvil	respuestas y operaciones pendientes hasta sincronización
Contratos técnicos	Repositorio Git	OpenAPI, DTO/schemas y documentación técnica

7. Offline y sincronización

La aplicación móvil debe permitir continuidad operativa en terreno sin conectividad. Las operaciones pendientes se conservan localmente y se envían al backend al recuperar conexión mediante POST /api/v1/sync/batch.

- Cada operación utiliza UUID y una idempotency_key.
- El backend puede responder accepted, duplicate, conflict o rejected.
- accepted y duplicate permiten retirar la operación de la cola local después de confirmación.
- conflict y rejected permanecen recuperables para revisión o corrección.
- La resolución final de conflictos debe respetar reglas institucionales y auditoría.

8. Seguridad

Control	Criterio arquitectónico
Autenticación	Acceso solo para usuarios autorizados; sesión gestionada mediante backend.
Autorización	RBAC validado en NestJS; el frontend no constituye una barrera de seguridad.
Transporte	HTTPS para comunicaciones cliente-servidor.
Credenciales	Nunca almacenar secretos reales en Git ni archivos .env versionados.
Auditoría	Registrar operaciones relevantes, cambios de estado, prioridad, correcciones y reevaluaciones.
Archivos	Acceso controlado a evidencia y documentos; la BD mantiene referencias y metadatos.
Protección adicional	Rate limiting, hashing robusto y 2FA/TOTP cuando corresponda a la definición institucional.

9. API y contratos

Método	Endpoint	Propósito
POST	/api/v1/auth/login	Autenticación
POST	/api/v1/inspecciones	Crear inspección
POST	/api/v1/inspecciones/{id}/respuestas	Registrar respuestas ECI
POST	/api/v1/inspecciones/{id}/evidencias	Registrar evidencia/metadatos
POST	/api/v1/inspecciones/{id}/calcular-ici	Cálculo ICI oficial 35/20/40/5
POST	/api/v1/inspecciones/{id}/correcciones	Corrección controlada y auditada
POST	/api/v1/inspecciones/{id}/reaperturas	Reapertura controlada
POST	/api/v1/inspecciones/{id}/reevaluaciones	Nueva inspección vinculada a la original
GET	/api/v1/mapa/inspecciones	Datos georreferenciados para visor
GET	/api/v1/dashboard/resumen	Indicadores agregados
POST	/api/v1/sync/batch	Sincronización offline idempotente

10. Arquitectura de despliegue inicial

Para el MVP académico se propone una arquitectura desplegable con servicios gratuitos o de código abierto, manteniendo independencia suficiente para migrar posteriormente a infraestructura institucional o financiada. El diseño no debe acoplar la lógica de negocio a un proveedor específico.

Capa	Despliegue conceptual inicial	Escalabilidad
Web	Hosting estático compatible con Vue/Vite	Puede migrarse a hosting institucional/CDN.
Mobile	Aplicación Flutter	Distribución controlada según definición MOP.
API	Servicio Node.js/NestJS	Contenerizable y migrable a infraestructura institucional.
BD	Supabase/PostgreSQL	PostgreSQL permite migración futura sin rediseñar el dominio.
Archivos	Object Storage S3-compatible	Escalable por capacidad y migrable entre proveedores compatibles.
IA	Servicio Python + proveedor IA	Adaptador desacoplado para sustituir proveedor.
Automatización	n8n	Flujos exportables/versionables y migrables.

11. Repositorio y gestión Kanban

La arquitectura se refleja en un monorepo con separación explícita de aplicaciones, servicios, contratos, integraciones, datos, pruebas y automatización de CI/CD.

Ruta / elemento	Contenido
apps/web	Vue 3 + TypeScript
apps/mobile	Flutter + Dart
apps/api	NestJS + TypeScript
services/ai	Servicio Python de IA auxiliar
contracts	OpenAPI, DTO y schemas
integrations	Object Storage, n8n y adaptadores
database	Migraciones y persistencia, sin fijar ORM mientras no esté seleccionado
tests	Pruebas unitarias, integración, E2E y contratos
.github	CI/CD, Pull Request templates y validaciones

La trazabilidad de implementación sigue: requisito/RF + prioridad → tarjeta Trello → rama → commits → Pull Request → pruebas/validación → evidencia.

12. Decisiones abiertas

- Proveedor definitivo de hosting y Object Storage para operación posterior al MVP.
- ORM o estrategia concreta de acceso a datos para NestJS.
- Estados y permisos institucionales definitivos.
- Política formal de retención de fotografías/documentos.
- Plantilla oficial de reportes y mecanismo de firma/validación.
- Canales de alertas y reglas definitivas de emergencia.
- Alcance final de IA y automatizaciones n8n.

13. Conclusión

La arquitectura propuesta permite construir el MVP solicitado manteniendo separación clara entre terreno, gestión, lógica de negocio, persistencia y servicios auxiliares. NestJS actúa como núcleo autoritativo; Flutter y Vue funcionan como clientes; PostgreSQL conserva información estructurada y Object Storage los binarios; MapLibre GL cubre el visor territorial; IA y n8n permanecen desacoplados. La solución queda preparada para evolucionar desde una fase académica gratuita hacia infraestructura institucional sin rediseñar el dominio funcional.